Day 1 Recap: Why Math Matters for AI

What machines actually are

Before any AI, before any algorithm, there's a basic fact worth sitting with: an electronic machine doesn't understand numbers, words, images, or anything else we care about. It understands one thing — electrical signals. Everything else is built on top of that, layer by layer.

The single invention that made modern computing possible is the **transistor** — a tiny electrical switch. Once you can reliably switch a signal on and off, you can build logic out of it, and once you can build logic, you can build everything else.

Binary: the first bridge

If machines only understand electrical signals, the first problem is: how do you represent *anything* — a number, a letter, a pixel — using just "on" and "off"?

The answer is **binary**: a mathematical model of those two electrical states. It's not the only possible choice, but it's the one that stuck, because it maps cleanly onto the physical reality of the hardware. Binary is the first of many layers of abstraction between the physics of a circuit and the problems we actually want to solve — layers we'll build up through this course.

Not everything stays inside the box: closure

Here's a question worth sitting with: if you take two numbers and add, subtract, multiply, or divide them, do you always get back a number of the same kind?

Try it with a few number systems and you'll find the answer is often *no*. This property — whether an operation keeps you inside the same set you started with — is called **closure**, and it fails more often than intuition suggests.

The same issue shows up in binary. Computers represent numbers using a fixed number of bits (a *data type*), which means only a finite range of values is representable. Add two numbers that are individually valid but whose sum falls outside that range, and you get **overflow** — a value that wraps around or produces a nonsensical result. If you've written code that produced a strange result from what looked like simple arithmetic, this is very likely why. It's the same closure problem, just showing up in a system you already have hands-on experience with.

Why we need formal mathematical tools

One more question worth asking directly: if we didn't have tools like linear algebra, calculus, and probability, how would you actually solve problems that need to run reliably, at scale, across millions of users and devices?

This is the real reason this course exists. It isn't "math for its own sake" — it's that scaling anything reliably (a machine, a model, a system) requires formal tools that guarantee consistent behavior, rather than ad hoc solutions that happen to work once. Linear algebra, calculus, probability, and the rest of this course's toolkit exist because they're *necessary*, not because they're traditional.

Three concrete illustrations

Why does data have structure? (Singular Value Decomposition)

A grayscale image is, mathematically, just a large matrix of numbers — a 768-row image would in principle need 768 numbers per column to describe exactly. But real images aren't random noise: using only the 50 most important "components" of that matrix (out of 768), you can reconstruct something almost indistinguishable from the original. That's not a compression trick specific to images — it's a general fact about real data: most of it has far less "true complexity" than its raw size suggests. The technique behind this, the Singular Value Decomposition, is something we'll build up to properly in a few weeks. The same underlying idea — finding the essential structure hiding in high-dimensional data — is also how face recognition, recommendation systems, and parts of large language models work.

Can you trust your intuition? (Bayes' theorem)

Here's a scenario: a disease affects 1% of a population. A test for it is 99% accurate. You test positive. What's the chance you actually have the disease?

Most people's intuition says something close to 99%. The real answer is closer to **50%**. Out of 10,000 people, 100 actually have the disease, and the test correctly flags 99 of them. But among the 9,900 healthy people, 1% — about 99 people — test positive anyway (false positives). So of the roughly 198 people who test positive, only about half actually have the disease. This is exactly why intuition alone isn't reliable when reasoning about uncertainty, and it's why Bayes' theorem — which we'll cover properly later in the course — matters well beyond this one example. It's the backbone of spam filters, medical diagnostics, and how language models reason about likely outcomes.

What powers ChatGPT?

When a model like ChatGPT reads a sentence — for example, "The cat sat on the mat because *it* was tired" — it has to figure out what "it" refers to. The mechanism that does this is called **attention**, and underneath, it's built from exactly the tools this course covers: vectors, matrices, and probability. We'll get to this properly in the course's final module. The point of mentioning it on day one is this: everything ahead in this course isn't abstract for its own sake — it's the actual machinery behind AI tools already in daily use.

What's ahead

The rest of the semester builds, in order: linear algebra, multivariable calculus, probability and clustering, regression and statistics, optimization, information theory, signal processing, and finally a module that ties all of it directly back to how modern generative AI works. Each module sets up the next — nothing here is arbitrary.